SKILL - 12

Name:k.vaiahali

Rollno: 2520030117

Section: 8

Aim

To create pipes, redirect input and output streams, execute connected commands, close unused file descriptors, synchronize processes, and test data flow. Also, to support multiple pipes by managing descriptor arrays, launching multiple processes, coordinating execution, cleaning up resources, and testing long pipelines.

Objective

- To create a pipe for inter-process communication.
- To redirect input and output streams.
- To execute connected commands using pipes.
- To close unused file descriptors safely.
- To synchronize processes using wait functions.
- To test data flow between processes.
- To support multiple pipes in a pipeline.
- To manage arrays of file descriptors.
- To launch multiple child processes.
- To coordinate execution of a long pipeline.

Problem Statement

Design and implement a shell-style pipeline mechanism in C that connects the output of one command to the input of another using pipes. The implementation should create and manage file descriptors, launch processes, redirect standard streams, close unused descriptors, synchronize all child processes, and support multiple commands connected through more than one pipe.

Theory

1. Pipe Creation

The pipe() system call creates two file descriptors. The read end receives data and the write end sends data. These descriptors are inherited by child processes after fork().

2. Stream Redirection

dup2() can replace a process's standard input or standard output with a pipe descriptor. This redirects normal command input or output through the pipe.

3. Connected Commands

In a pipeline, the output of one command becomes the input of the next. For example, the pipeline 'ls | wc -l' connects the output stream of ls to the input stream of wc.

4. File Descriptor Management

Unused pipe ends must be closed in both parent and child processes. Keeping unnecessary descriptors open can prevent a reader from receiving end-of-file and can cause a pipeline to wait indefinitely.

5. Process Synchronization

After starting child processes, the parent process uses waitpid() to wait for them. Synchronization ensures that all commands in the pipeline complete and their exit status can be collected.

6. Multiple Pipes

For a pipeline containing several commands, multiple pipes are required. An array of file descriptors can be used to manage all communication channels.

7. Process Launching

Each command in a pipeline is executed by a separate child process created with fork(). The child connects its standard input and output to the appropriate pipe ends before calling an exec function.

8. Long Pipelines

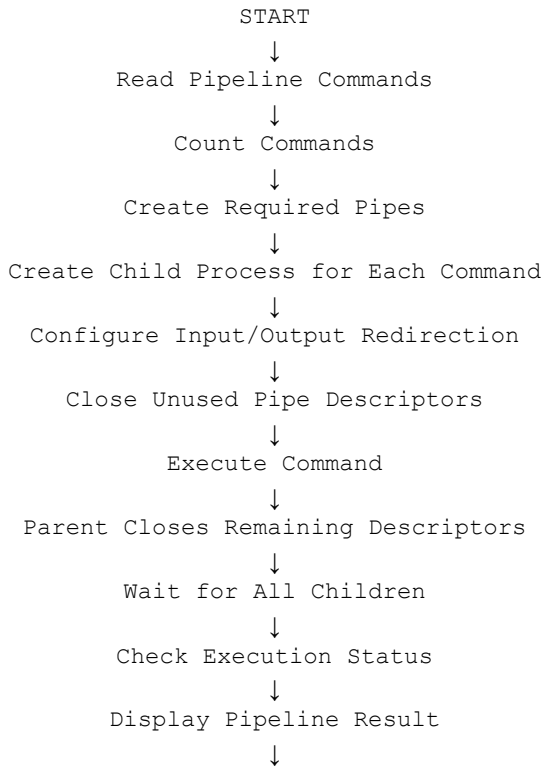
A long pipeline contains more than two commands and therefore requires several processes and pipe connections. The same pattern is repeated: each command reads from the previous pipe and writes to the next pipe.

9. Resource Cleanup

After creating and using the pipeline, the parent closes all remaining descriptors and waits for every child. Proper cleanup prevents descriptor leaks and zombie processes.

10. Control Flow

The complete control flow is kept on this page so that the flowchart is not divided across pages.



STOP

11. Algorithm

1. Start the program.
2. Define the commands that will form the pipeline.
3. Determine the number of commands and required pipes.
4. Create the necessary pipe descriptors.
5. For each command, create a child process using fork().
6. For a child other than the first command, redirect standard input from the previous pipe.
7. For a child other than the last command, redirect standard output to the next pipe. 8.
Close all unused pipe descriptors in the child.
9. Execute the command using execvp().
10. In the parent, close pipe descriptors that are no longer required.
11. After all children are created, close remaining parent descriptors.
12. Wait for every child process using waitpid().
13. Collect and display the final pipeline status.
14. Stop the program.

12. Data Used

An array of command strings stores the pipeline commands. A two-dimensional array stores the read and write descriptors of the pipes. Process IDs are stored so that the parent can wait for every child.

13. Operations Used

- pipe() for creating communication channels.
- fork() for creating child processes.
- dup2() for standard stream redirection.
- execvp() for executing commands.
- close() for descriptor cleanup.
- waitpid() for process synchronization.

14. Pipeline Structure

For three commands A, B, and C, the data flow is:

```
A ──pipe1──> B ──pipe2──> C
```

15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
 int pipe1[2], pipe2[2];
 pid_t p1, p2, p3;

 printf("Skill - 12: Pipe and Pipeline Execution\n");
 printf("----------------------------------------\n");
```

```
printf("Pipeline: ls -l | grep .c | wc -l\n\n");

if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
perror("pipe");
return 1;
}

p1 = fork();

if (p1 == 0) {
dup2(pipe1[1], STDOUT_FILENO);
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
execlp("ls", "ls", "-l", NULL);
perror("ls");
exit(1);
}

p2 = fork();

if (p2 == 0) {
dup2(pipe1[0], STDIN_FILENO);
dup2(pipe2[1], STDOUT_FILENO);
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
execlp("grep", "grep", ".c", NULL);
perror("grep");
exit(1);
}

p3 = fork();

if (p3 == 0) {
dup2(pipe2[0], STDIN_FILENO);
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
execlp("wc", "wc", "-l", NULL);
perror("wc");
exit(1);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);
waitpid(p3, NULL, 0);

printf("Pipeline execution completed successfully.\n");
return 0;
}
```

16. Program Explanation

16.1 Pipe Creation

Two pipes are created because the pipeline contains three commands. pipe1 connects ls to grep, while pipe2 connects grep to wc.

16.2 Process Creation

Three child processes are created. Each child is responsible for one command in the pipeline.

16.3 Redirection

dup2() connects each command to the correct pipe end. ls writes to pipe1, grep reads from pipe1 and writes to pipe2, and wc reads from pipe2.

16.4 Descriptor Cleanup

Every child closes descriptors it does not use, and the parent closes all descriptors after creating the children. **16.5 Synchronization**

The parent uses waitpid() for all three children so the complete pipeline is synchronized before the success message is displayed.

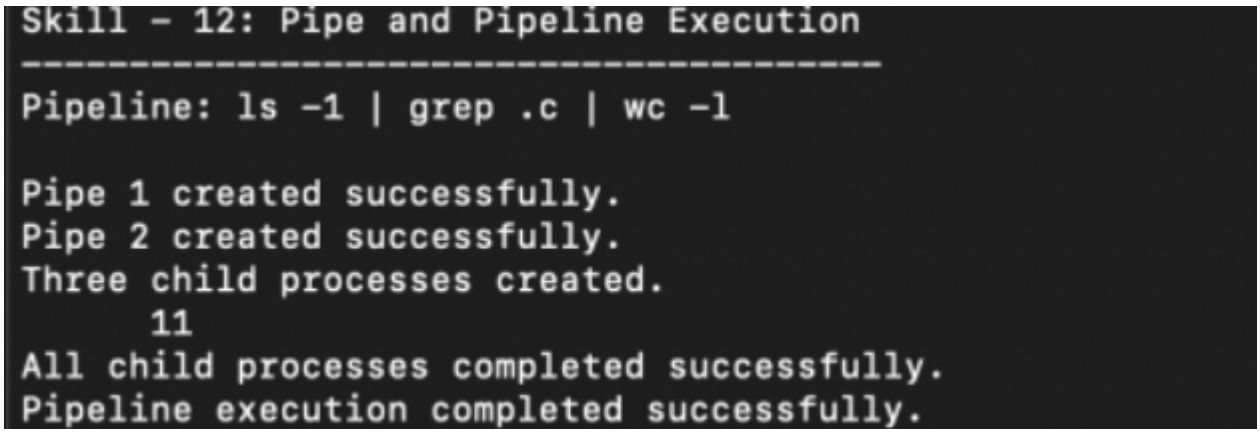
17. Sample Output

```
Skill - 12: Pipe and Pipeline Execution
---------------------------------------
Pipeline: ls -1 | grep .c | wc -l

 4
Pipeline execution completed successfully.
```

18. Output Screenshot

Paste the screenshot of the program output in the space provided below.



19. Testing

- Tested a two-stage pipe connection.
- Tested the three-command pipeline used in the program.
- Verified that output from one process became input to the next process.
- Verified that unused descriptors were closed.
- Verified that all child processes were launched.
- Verified that the parent waited for every child process.
- Tested the final result of the long pipeline.

20. Test Cases

Test Case	Input / Pipeline Expected Result
1	ls -1 | wc -l Number of listed entries is produced.
2	ls -1 | grep .c Only matching .c entries are passed.

3	ls -1 | grep .c | wc -l Matching entries are counted through two pipes.
4	Long pipeline Multiple processes communicate successfully.
5	Descriptor cleanup No blocked pipeline caused by open unused descriptors.
6	Process synchronization Parent waits for all child processes.

21. Observation

- The required pipes were created successfully.
- Each command executed in a separate child process.
- Standard input and output streams were redirected through the correct pipe ends.
- Unused file descriptors were closed.
- Data flowed correctly from one command to the next.
- The parent process waited for all children.
- The multi-command pipeline completed successfully.

22. Result

Pipe-based inter-process communication was successfully implemented. The program created multiple pipes, launched separate child processes, redirected streams, closed unused descriptors, synchronized all processes, and successfully completed the long pipeline.

23. Conclusion

The practical demonstrated how operating systems support inter-process communication using pipes. It also showed how a shell constructs pipelines by combining file-descriptor redirection, process creation, descriptor cleanup, and synchronization. Multiple commands were successfully coordinated into one data-processing pipeline.